

The Experience

How Proxy looks, feels, and shows up in the meeting. Part A is the product — the whole experience and every feature a person actually touches. Parts B–D are the build — the one rendering mechanism everything rides on, each surface component with how we wire and optimize it, and why rendering-into-video means no per-platform UI and no adoption friction.

Everything in the other deep dives — the estate map, the two-speed understanding, the actions, the verifiers — has to reach a real person sitting in a real meeting. This is how. Proxy shows up as a participant you can see and talk to, not a dashboard you go visit. And it is deliberately quiet: in a category of loud, purple-gradient AI, Proxy is the calm, confident one, and restraint is the brand.

THE ONE IDEA TO HOLD

Proxy is **operated by talking and glancing** — nothing else. You speak to it or type one line; you glance at its tile. If a feature needs more than that — a second app to install, a panel of knobs, a graph to navigate mid-meeting — it is *too much* and it is cut. Everything Proxy shows renders into **one canvas** that is small when it's its video tile and big when it promotes to screen-share, so it is identical on Zoom, Meet, and Teams because to the platform it is just video. That single constraint and that single mechanism explain every choice below.

PART A — THE PRODUCT

The whole experience, and every feature in it

What a person sees, hears, and does — written before any implementation detail.

Proxy is invited to a meeting like a person, joins it, and behaves like the calmest, best-prepared colleague in the room. You never open an app to use it. There are exactly four places it reaches you, and three of them are things the meeting already has: its **video tile** (which it draws itself, and can grow to a screen-share), the **meeting chat** (the lane you type to it in), and — after the meeting — a **Slack or Teams message** with your items. A minimal companion link exists only for the rare thing that needs a real click or true privacy. That is the entire surface footprint, on purpose.

The governing constraint: talking and glancing

Every design decision is measured against one test: *can I use this without stopping paying attention to the meeting?* If operating a feature makes you look away from the other people and fiddle with software, it fails, and it is removed. This is why there is no Proxy app to learn, no 24/7 outside assistant, and no wall of settings. You address Proxy the way you'd address a person — by speaking, by @-mentioning it, by typing a sentence — and you read its state the way you read a colleague's face — with a glance. The "too much" line is a real cut we enforce, not a slogan: it is what keeps the product from bloating into another thing on your screen you have to manage.

Identity — the calm one

Proxy has no face and no avatar. It is a single living **presence-orb**: a soft geometric bloom that breathes. A humanoid avatar would be uncanny; a static logo would be cold. Because there's no face, its entire personality lives in **motion** — and the motions are legible from across the room without a word:

- **Slow breath** — it is listening.
- **Lean toward the speaker** — it is attentive to whoever is talking right now.
- **Quick shimmer** — it is checking something (running a query, verifying a number).
- **Firm, calm settle** — it has something to say and is holding it, available but not interrupting.
- **Gentle rise** — it has raised its hand.
- **Visible sit-back-down** — it is yielding the floor.

Its signature is a warm, confident **deep teal-ink** — deliberately not the electric-purple AI cliché. The copy voice matches the visual one: plain, specific, warm; it says what it did and shows the receipt; never "As an AI," no filler, no exclamation-point enthusiasm. And there is a hard ceiling on the expression: a slow rotation when thinking, a slight waver when genuinely uncertain (calibrated confidence, made visible), a steady brighten when confident, a soft bloom when it finds something — and nothing more. No facial features, no theatrics. More than the waver-lean-bloom vocabulary would be a costume, not an identity.

ONE VOICE, ALWAYS — AND HOW A SYSTEM "SPEAKS FOR ITSELF"

Sometimes the natural thing is for a specific system to talk in the first person: "checkout here — my p99 is 340ms." When that happens, the **tile promotes**: the orb keeps Proxy's exact form but takes on a subtle accent of that system's colour and renames to it ("checkout"), and the first person lives entirely *in the words*. The audio is still the one calm Proxy voice. We deliberately do **not** give each system its own synthetic voice — even setting cost aside, many voices in a call confuse who is speaking and read as a gimmick, while the visual promotion plus first-person phrasing already deliver "the software speaks for itself" with one coherent identity. One consistent mark — orb plus wordmark — appears on the tile, in chat, on the Slack/Teams pings, and on the closing reveal, so Proxy is one recognizable presence on every surface. Its resting state is always the calm breathing orb: present, never demanding.

Presence — a real participant on the canvas

Because Proxy's tile is a canvas it fully controls, it can behave on screen the way a person does. It **shares its screen like a colleague** — pulls up the actual dashboard, the actual code, the actual test run or document — and it moves a **visible cursor** you watch scroll, click, and highlight (the cheapest trick there is for making a screen-share feel human, not automated). It **circles and pins the exact line or number** it means, so "this one" is literal, not a paragraph of description. It lets **work form in front of you** — a model rebuilding, a mockup rendering, a timeline assembling — instead of hiding behind a spinner. It can **whiteboard**: sketch an architecture live, drawing the boxes and arrows as it explains them, which is uniquely possible because it is driving a canvas rather than pasting a finished image. It drops lightweight **reactions** — a ✓, a nod-equivalent — as non-verbal acknowledgment, and shows a "listening to Maya" cue so it is never a dead box in the grid.

Crucially, Proxy promotes its canvas to the *shared* screen only when seeing the work actually helps the room — a test running, a model rebuilding, a diagram worth everyone's eyes. When the work is minor it stays small on Proxy's own tile. It takes the shared screen the way a good colleague does: when there is something worth looking at, never to narrate every keystroke.

How it speaks up

Proxy's manners are encoded, because a proactive AI that acts on its own is only welcome if it is socially safe. Four behaviours carry that:

- **Silent bid-to-speak.** When Proxy has something, the orb *firms* and shows a one-line reason — no interruption, just an available signal you can take or ignore.

- **Whisper-first.** A sensitive catch goes to the *addressed person privately first*, before the room hears it — "dashboard says 3.4%, not 2% — want me to share?" — and they decide whether to surface it. This is the single etiquette that makes proactive correction safe to turn on: Proxy never corrects you in front of your team without giving you the first move.
- **Raise a hand.** A material catch (something the room genuinely needs) raises Proxy's hand with its reason attached, so the interruption is legible and earned, not a random blurt.
- **Texture tells you the type.** A contribution's kind is readable before you read the words: a *verified* assertion carries a solid edge and its receipt; a *judgment* is dashed; an *offer* is a dashed outline. You know at a glance whether Proxy is stating a fact, giving an opinion, or offering to do something.

And what Proxy chose *not* to say is not lost — every silence, with its reason, is priced into the close-of-meeting reveal.

Control — the dial, and mute

The entire front-line control surface is one **five-level dial**: Observer → Notetaker+ → Contributor → Active → Facilitator. Turning it up doesn't just change a setting in the dark — it **previews live** what more forwardness would surface, showing you the held-back items that a higher setting would let through, so you decide with evidence. A one-tap **mute** / **quiet-down** instantly drops Proxy to Observer the moment a conversation turns sensitive. That's it — a dial and a mute. Per-agent modes (silent, text-only, raise-hand, speak-when-addressed, interject, plus an authority level) exist as an *advanced* setting for people who want fine control, but they all resolve to the same one dial threshold underneath. There is never a second, competing control system to learn.

In-meeting helpers everyone wants

These ride the tile and the chat, cost almost nothing, and are the things any team wishes someone in the room would just do:

- **Timekeeper** — quietly tracks the agenda against the clock: "10 minutes left, 2 items open." The role nobody wants to play.
- **Decision-marking** — stamps a decision unambiguously on the tile the moment it lands: "decision logged: ship Friday." Kills the "wait, did we actually decide that?" a week later.
- **Action-capture** — action items accrue visibly on the tile through the meeting and are finished at close, so nothing falls through.

- **Who-hasn't-weighed-in** — on the Facilitator setting, surfaces the quiet expert before the room decides without the one person who knew.
- **Catch-me-up** — a ~20-second recap for a late joiner, either on the tile or whispered privately in chat.

Join and consent

Proxy is invited like a person — by calendar or email — and it **belongs to the room, not the host**: anyone present can address it and anyone gets its help, not just whoever set it up. On join it states plainly, in one line, *what it can see* in this particular room and *where its dial sits*. Its permission scope is always one glance away and is **default-deny** — it can only ever see what the people in the room could already open themselves. A bot that reads a company's software and can take actions is only welcome if what it sees and what it may do is never a mystery, so we make both glanceable by default rather than buried in a settings page.

The lobby — before the meeting starts

As the meeting opens, "meet your Proxy": the **cast** of systems it expects to need today (the checkout expert, orders and invoicing on standby), the **dial preset** for this meeting type, and the **context** it was handed. The relevant experts sync to the agenda — the pre-meeting recheck from Deep Dive 01 rendered as a visible team huddle — so Proxy walks in known, not cold, and you can see the society of agents lining up behind the one orb before anyone has spoken.

Close of meeting — the reveal

This is the emotional peak and the artifact that sells the next meeting: a shareable **receipt**. "Verified 11 claims, caught 2 problems, ran 1 test nobody asked for, stayed quiet 6 times — here's why." Every count is expandable to its evidence, and every silence names its reason. Alongside it, the staged drafts made during the meeting collect into one **draft bundle** you approve in a single pass instead of hunting for a ticket here and a PR there; Proxy proposes **what it would do next** as one-tap follow-ups; and each person's items are prepared to ping them afterward.

Adoption — before you ever rely on it

The scariest moment in the whole product is the first one: an AI is about to read your company's code. We turn that into a trust spectacle rather than a leap of faith. A **living graph explorer** lets you watch the estate map assemble itself, browse any system's page, and see the honest gaps lit up —

the things Proxy admits it doesn't know yet. And a **self-test ritual** has Proxy generate questions about your own estate, answer them, and show its work verifying each answer against the live source — so you watch it earn trust before you depend on it. (The graph explorer is a reference-and-adoption tool, deliberately *not* something you navigate mid-meeting — that would violate the glance rule.)

HONEST DEFERRALS — WHAT IS NOT IN THE FIRST VERSION

Progressive trust ("earn the dial up"). The eventual answer to "why switch on an autonomous AI?" is that you never have to: Proxy arrives quiet, low on the dial, and earns its way up as its visible track record proves out, room by room. The pieces this rests on ship in v0 — shadow mode and a dial preset by meeting type — but the productized *earn-your-autonomy promotion flow* is **post-v0**.

A standalone Workspace / 24-7 assistant is cut. Everything lives inside the meeting. A separate persistent app was cut for two honest reasons: the platforms (Zoom, Meet, Teams) don't let us inject rich UI outside the call, and a separate app has fatal adoption friction — no one would open it. The one out-of-meeting touch we keep is the delivery ping (below), which is delivery of meeting work, not a standing relationship.

PART B — HOW IT WORKS

Four surfaces, one rendering mechanism

The tile-canvas explains almost everything. Here it is in a picture, then in words.

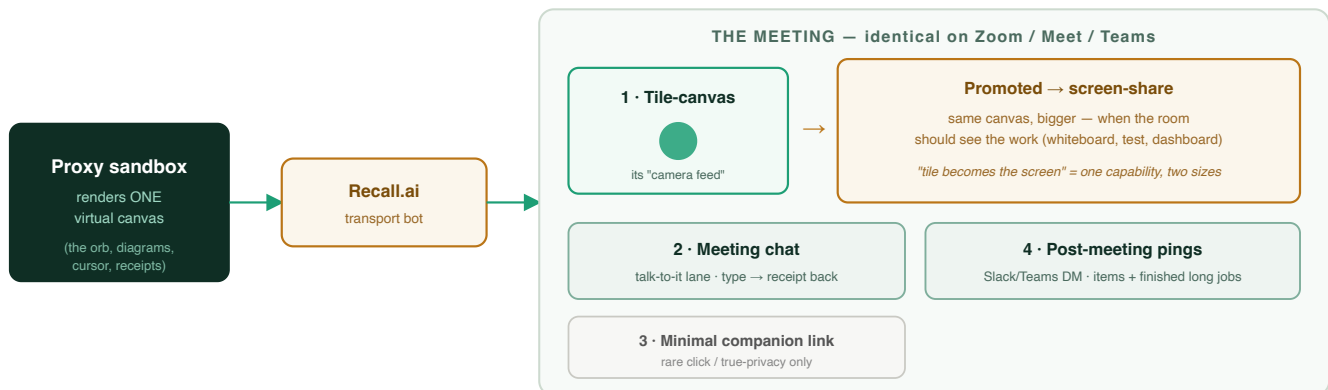


Figure 1 — The four surfaces. Proxy renders one virtual canvas in its sandbox and streams it through the Recall.ai transport bot as its camera feed. Small, it is the tile; promoted, it is a screen-share — the same canvas at two sizes. The chat is the talk-to-it lane, the companion link is a rare fallback, and the only out-of-meeting touch is a delivery ping.

The tile-canvas — the primary surface

Here is the whole trick, and it is what makes the rest possible. A meeting bot that joins as a participant can output its own video and audio, and can screen-share, through the **Recall.ai** transport layer (the same bot that captures the meeting audio in Deep Dive 02). So Proxy renders *whatever it wants* to a **virtual canvas** inside its sandbox — the breathing orb, a diagram, a moving cursor, a receipt flashing, "listening to Maya," the decision-logged stamp — and streams that canvas out as its camera feed. To Zoom, Meet, or Teams it is just a participant's video. That means:

- **Any rich UI we can draw, we can show** — because it's just pixels in a video frame, we are not limited to whatever widgets a platform's bot API happens to expose. A whiteboard, a forming mockup, a highlighted line of code: all just things drawn on the canvas.
- **Small is the tile; big is the screen-share.** "Proxy's tile becomes the screen" and "Proxy shares its screen" are the *same capability at two sizes*. Proxy picks the size for the moment — small on its tile for minor work, promoted to a screen-share when the room should see it.
- **It is identical on every platform** because none of it depends on platform-specific UI. A video frame is a video frame on Zoom, Meet, and Teams alike.

The meeting chat — the talk-to-it lane

The second surface is the meeting's own chat. You type a sentence, Proxy computes it against the estate, and it answers with a receipt — behaving identically to asking out loud, except the answer comes back as text instead of speech. On Zoom this can be a genuine **private direct message** to Proxy; where a platform lacks native private DM, we use a light fallback so the lane still works everywhere. This is also the surface the whisper-first catch and the catch-me-up recap arrive on when they need to be private.

The companion link and the post-meeting ping

The **minimal companion link** is the deliberate exception: an optional link for the rare case that genuinely needs a real click or true privacy that chat can't give. It is near-nothing by design — not a place you live, just a door for the edge case. And the **post-meeting ping** is the single out-of-meeting touch: after the meeting, each person gets a Slack or Teams message with their action items and, when a long-running job finishes after the meeting ends, its result — with one-click actions built in. This is delivery of meeting work, not a standing assistant relationship.

Every surface component — what it is, how we wire it, how we optimize it

For each piece: first the overview in plain terms, then the technical specifics and the optimization. Dense, but built to implement from.

Component 1 — The virtual canvas and the transport

Overview. The canvas is a headless rendering surface inside Proxy's sandbox — think of it as a screen that no monitor is attached to, that Proxy draws onto and we stream out as video. Everything visual Proxy does is a thing drawn on this one canvas.

How we wire and optimize it. We render into a headless browser context in the same sandbox the actions run in (Deep Dive 05), so the canvas can display anything a web page can — HTML, SVG, live-updating charts, a driven browser tab showing a real dashboard. The rendered frames are handed to the **Recall.ai output-media API**, which pipes them into the meeting as Proxy's camera or screen-share stream. The one real engineering unknown here is *frame rate* — how smoothly the canvas animates through the transport — which is exactly why the build order front-loads a **one-day spike** on the Recall output-media API before committing to the richest animations; the capability itself is established, only the smoothness needs measuring. We optimize by rendering at a modest, steady frame rate (this is calm ambient motion, not a video game — the orb breathes, it doesn't sprint), and by promoting to full screen-share resolution only when Proxy actually shares, keeping the tile stream light the rest of the time.

Component 2 — The presence-orb renderer

Overview. The orb is the identity, and its whole vocabulary is motion. This component turns Proxy's internal state (from the two-speed understanding engine in Deep Dive 03/04) into the breath, lean, shimmer, settle, rise, and sit-back-down you read on the tile.

How we wire and optimize it. The orb is a small set of animated shapes bound to a *state machine* — a fixed list of states (listening, checking, has-something, hand-raised, yielding, uncertain, confident) each with one defined animation. The understanding engine already emits these states as events; the renderer just subscribes and plays the matching motion. That keeps it cheap and, more importantly, *honest*: the waver that signals uncertainty is driven by the actual calibrated-confidence

value the verifiers produce, not a decorative flourish. The hard ceiling (no face, no theatrics) is enforced in the component itself — there simply is no code path to render a facial feature — so the identity can't drift into a costume as features accrete. Tile **promotion** (orb keeps its form, takes the speaking system's colour accent, renames to it) is a state transition on this same machine, which is why it ships day one and is purely visual.

Component 3 — Presence behaviours (cursor, pin, whiteboard, forming work)

Overview. These are the behaviours that make a screen-share feel like a person is driving it: the visible cursor, the circle-and-pin on the exact line, the live whiteboard, the artifact forming instead of a spinner.

How we wire and optimize it. The visible cursor is a drawn overlay on the canvas whose position we animate along a path — cheap, and the single biggest perceived-humanity gain per line of code. **Pin/circle** reuses the pin-to-referent mechanism from the understanding engine: Proxy already knows the exact file:line or the exact cell a claim points at, so highlighting "this one" is drawing a ring at a coordinate it already resolved, not new intelligence. The **whiteboard** is Proxy drawing SVG boxes and arrows onto the canvas as it narrates — uniquely possible precisely because it drives a canvas rather than pasting a static image. **Work forming** means we render the artifact's intermediate states as they're produced (a chart's bars filling, a mockup's blocks landing) instead of a loading spinner — this is free because the work is happening anyway; we just show it. The optimization across all of these is that *none of them is new computation* — they render exhaust the system already produces, which is why the whole presence layer adds no meaningful cost.

Component 4 — The speak-up relay (bid, whisper, hand-raise, texture)

Overview. This renders Proxy's social manners: the silent bid-to-speak, the whisper-first private catch, the material hand-raise, and the confidence texture that tells you the contribution's type at a glance.

How we wire and optimize it. The proactive engine (Deep Dive 04) decides *whether* and *how forcefully* to surface something; this relay decides *where it lands*. A low-stakes bid firms the orb and shows a one-line reason on the tile. A **sensitive catch is routed to the meeting chat as a private message to the addressed person first** — the relay checks the catch's sensitivity flag and, if set, will not render it to the room at all until that person chooses to surface it. A material catch triggers the hand-raise state on the orb. **Texture** is a rendering rule bound to the verifier's output class: verified → solid edge, judgment → dashed, offer → dashed outline. The optimization is that

sensitivity and confidence are already computed upstream — the relay is pure routing and styling, so the socially-critical whisper-first guarantee is enforced by a single, testable branch rather than smeared across the system.

Component 5 — The dial and mute

Overview. The one control surface: a five-level dial with a live preview of what turning it up would surface, and a one-tap mute that drops Proxy to Observer.

How we wire and optimize it. The dial is a single threshold value that the proactive engine reads before surfacing anything — one number, not a panel of independent switches. The **live preview** works because the proactive engine already holds a queue of items it considered and held back, each tagged with the dial level that would release it; raising the dial in the preview simply shows the ones that would now clear the threshold. **Mute** writes the Observer value to that same threshold instantly. The advanced per-agent modes (silent / text-only / raise-hand / speak-when-addressed / interject, plus authority) are a front-end that *resolves down to the same single threshold* — so there is exactly one control variable in the system, and everything else is a nicer way to set it. That is the anti-inflation guarantee made structural: we cannot accidentally ship a second control system because there is only one value to set.

Component 6 — The meeting-chat compute lane

Overview. Typing to Proxy in chat is a first-class way to use it — you type, it computes against the estate, it answers with a receipt.

How we wire and optimize it. A typed message enters the *same* reactive path as a spoken ask (Deep Dive 03) — the only difference is the return surface (text in chat vs. speech in the room). On Zoom we use the native private-DM channel to the bot; elsewhere we fall back to a light lane that preserves the same behaviour. This is why chat costs almost nothing to support: it is not a separate feature, it is the existing reactive engine with a different input and output socket. The whisper-first catch and the catch-me-up recap reuse this same private lane.

Component 7 — In-meeting helpers

Overview. Timekeeper, decision-marking, action-capture, who-hasn't-weighed-in, catch-me-up. All render onto the tile or the chat; all are cheap.

How we wire and optimize it. Each helper is a small reader over data the understanding engine already produces. The **timekeeper** reads the agenda and the clock and stamps "10 min left, 2 items

open" on the tile. **Decision-marking** subscribes to the decision-crystallized event and stamps it on the tile. **Action-capture** accrues items on the tile as they're detected and finalizes them at close. **Who-hasn't-weighed-in** reads the observable speaking dynamics (who has and hasn't spoken to an open decision) and, on the Facilitator dial, surfaces the quiet expert. **Catch-me-up** renders a ~20-second recap from the living page, either on the tile or whispered in chat to the late joiner. None of these is new intelligence — they are renderings of exhaust — which is why they're all cheap and ship together.

Component 8 — Join, consent, and the lobby

Overview. Proxy joins like a person and belongs to the room; on join it states what it can see and its dial; before the meeting the lobby shows the cast, the preset, and the context.

How we wire and optimize it. Proxy joins from a calendar or email invite via the same Recall.ai transport. "Belongs to the room, not the host" is enforced by the permission model (Deep Dive 08): access is scoped to what the people present could already open, filtered at query time, default-deny — so *anyone* in the room can address it and get help within that scope, and no one can make it reveal what they couldn't already see. The one-line "here's what I can see and where my dial sits" is rendered from the permission scope and the dial value. The **lobby** is the pre-meeting recheck (Deep Dive 01) rendered as a huddle: it reads the agenda, resolves which experts are likely needed, warms their caches, and shows the cast, the dial preset, and the handed context on the canvas before anyone speaks. The optimization is that the lobby is *already happening* as a pre-meeting warm-up; we're just making the invisible warm-up visible.

Component 9 — The reveal, the bundle, and the pings

Overview. The closing receipt, the one-pass draft bundle, the proposed next steps, and the per-person post-meeting delivery.

How we wire and optimize it. The **reveal** is assembled from the audit trail (Deep Dive 08), which already logs every verified claim, every catch, every action, and — critically — every silence with its reason. So "verified 11, caught 2, ran 1 test nobody asked for, stayed quiet 6× — here's why" is a query over the log, rendered as a shareable card, not a new tally we maintain separately. The **draft bundle** gathers the drafts staged during the meeting (a ticket, a PR, an email) into one card approved in a single pass. **What-it-would-do-next** is the proactive engine's proposed follow-ups as one-tap accepts. The **post-meeting pings** go out over Slack/Teams to each named person with their items and any finished long-job result, with the one-click actions built in. The optimization

throughout: the reveal is the log made legible — its honesty (it can't overstate, because it's reading the record) is exactly what makes it a trustworthy sales artifact.

Component 10 — The adoption surfaces (graph explorer, self-test)

Overview. Before anyone relies on Proxy: a graph explorer to watch the estate map assemble and see the honest gaps, and a self-test ritual to watch Proxy earn trust against live source.

How we wire and optimize it. The **graph explorer** renders the Estate Graph (Deep Dive 01) as a navigable UI — click a system, see its page, owners, connections, and the known-unknowns lit up as honest gaps. This is the one place a graph is navigated, and it is deliberately a *reference and adoption* tool, never something used mid-meeting (that would break the glance rule). The **self-test** renders the training exam (Deep Dive 01) as a watchable spectacle: Proxy generates questions about your estate, answers them, and shows its work verifying each against the live source. The optimization is that both are re-renderings of mechanisms that already exist for other reasons — the graph is the estate map with a UI on top; the self-test is the training exam shown to a human — so the highest-stakes trust moment costs us almost no new build.

Why rendering-into-video means no per-platform UI and no adoption friction

The whole surface layer scales for one structural reason: **we render into video, so we never build per-platform UI.**

- **No per-platform UI, ever.** Because every rich surface is drawn onto one canvas and streamed as a video frame, there is nothing platform-specific to build. Zoom, Meet, and Teams each receive the identical stream. A new platform is supported the moment Recall.ai's transport reaches it — we write zero new UI. Contrast the alternative (a real integration into each platform's own UI extension model), which would be three separate, permission-gated, constantly-shifting codebases; we have one canvas.
- **No adoption friction.** There is nothing to install and nothing to learn. Proxy is invited like a person and operated by talking and glancing. The single hardest thing about adopting an in-meeting AI — getting a whole team to install and open a new app — is eliminated by never having one. The one out-of-meeting touch (the delivery ping) lands in Slack or Teams, where people already are.
- **Almost no new cost or latency.** Screen-presence is the only moderate lift, and it rides the sandbox and driven browser already in the stack for actions. Every other surface renders *exhaust the system already produces* — coverage, silence-reasons, receipts, states, the graph — so the surface layer is surfacing, not new thinking. The cost note holds: no meaningful cost or latency is added by this section.
- **One identity everywhere.** The single orb-plus-wordmark mark renders identically on the tile, in chat, on the pings, and on the reveal. As surfaces multiply, the identity does not fragment, because it is one renderer driven by one state machine — the anti-costume ceiling scales with the product.

Done when

The experience is complete when: every rich surface renders into the tile-canvas (or its promoted screen-share) and reaches Zoom, Meet, and Teams identically with no required install; a participant can operate Proxy with only a glance or a spoken/typed sentence, and any feature failing that test

has been cut; the meeting chat computes an answer with a receipt and behaves identically to a voice ask apart from the return surface; the presence-orb's state is legible across the room by motion alone, with no facial features anywhere in the renderer; a sensitive catch reaches the addressed person privately before the room, with no leak; the dial and mute are the entire front-line control, with per-agent modes resolving to the same single threshold; on join Proxy states what it can see and its dial, its permission scope is glanceable and default-deny, and anyone in the room — not only the host — can address it; the reveal reproduces the verified count, the catches, the unasked work, and the priced silences with their reasons, straight from the audit log; the post-meeting pings reach the right people with working actions, including finished extended jobs; the lobby shows the cast, the preset, and the context before anyone speaks; the adoption graph explorer and self-test let a buyer watch trust get earned before any meeting; and the identical surface layer serves every meeting type, with only the estate, the cast, and the dial differing — while the earn-your-autonomy promotion flow and any standalone workspace remain honestly out of the first version.